



Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université De Carthage
Institut National des Sciences Appliqués et de Technologie
4^e année Informatique Industrielle et Automatique

Projet de fin d'année

Système de supervision de santé de la machine basé sur l'IoT et l'IA.

Réalisé par :

- Mrad Rayen
- Fandouli Elyes

Encadré par :

- Mme. Ben Ali Khawla
- Examiné par :
– Mr. Hamdi Med. Ali

Soutenu le :
02/06/2026

Année Universitaire :
2025/2026

Remerciements

Nous souhaitons exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce rapport. Nous tenons à remercier particulièrement notre encadrante Mme. Benali Khawla pour son soutien continu tout au long de ce projet. Nous exprimons aussi notre reconnaissance envers tous ceux qui ont partagé leurs connaissances et leur expertise, nous permettant ainsi de mener à bien cette étude. Votre aide précieuse a été un pilier fondamental pour le succès de notre travail en binôme.

Nous remercions également l'équipe pédagogique de la filière Informatique Industrielle et Automatique pour la formation reçue dans les domaines de l'électronique, des réseaux, de la programmation embarquée et de la supervision industrielle. Les connaissances acquises durant le cursus ont constitué une base essentielle pour concevoir ce système de surveillance de santé machine.

Résumé

Ce rapport présente la conception et la réalisation d'un système de surveillance de santé machine basé sur l'Internet des Objets industriel. La solution proposée repose sur un microcontrôleur microcontrôleur doté d'une connectivité Wi-Fi intégrée, associé à un ensemble de capteurs mesurant le bruit, les vibrations, la présence de flamme, la pression atmosphérique, la température et l'humidité. Les mesures sont acquises localement puis transmises vers un broker MQTT cloud afin d'alimenter un dashboard temps réel.

Ce Projet de Fin d'Années présente la première étape vers un univers plus grand de l'IoT et l'IIoT. L'architecture proposée suit une approche IIoT en couches : perception, communication, cloud, application et analyse.

Ce projet vise à concevoir un système de supervision complète sur la santé de la machine permettant d'inspecter et planifier les maintenances nécessaires pour assurer la qualité des produits introduits dans la chaîne de production.

Glossaire

microcontrôleur : Microcontrôleur intégrant un processeur double coeur, des GPIO, des ADC et une connectivité Wi-Fi/Bluetooth.

IoT : Internet des Objets, ensemble de dispositifs connectés capables de mesurer, traiter et transmettre des données.

IIoT : Internet Industriel des Objets, application de l'IoT aux environnements de production et de maintenance.

MQTT : Protocole léger de messagerie fondé sur le modèle publication/abonnement.

Broker : Serveur MQTT responsable de recevoir les messages et de les redistribuer aux clients abonnés.

Dashboard : Interface de visualisation permettant de suivre les mesures, les tendances et les alertes.

FreeRTOS : Système d'exploitation temps réel léger permettant d'organiser le firmware en tâches concurrentes.

GPIO : Broche d'entrée/sortie générale utilisée pour connecter des capteurs et actionneurs.

ADC : Convertisseur analogique-numérique transformant une tension en valeur numérique exploitable.

I2C : Bus série synchrone à deux fils utilisé pour communiquer avec des capteurs comme le BMP280.

BMP280 : Capteur numérique de pression atmosphérique et de température.

DHT11 : Capteur numérique économique de température et d'humidité relative.

FFT : Transformée de Fourier rapide utilisée pour analyser les composantes fréquentielles d'un signal vibratoire.

QoS : Niveau de qualité de service MQTT indiquant la garantie de livraison d'un message.

TLS : Protocole de chiffrement recommandé pour sécuriser les communications réseau.

Table des matières

Remerciements	i
Résumé	ii
Glossaire	iii
1 Introduction, état de l’art et analyse des besoins	1
1.1 Introduction Générale	1
1.1.1 Contexte industriel : Industrie 4.0 et maintenance intelligente	1
1.1.2 Problématique	2
1.1.3 Objectifs du projet	2
1.1.4 Plan du rapport	2
1.2 État de l’Art	3
1.2.1 Maintenance préventive et maintenance prédictive	3
1.2.2 IoT industriel	4
1.2.3 Systèmes de monitoring existants	4
1.2.4 Protocoles IoT : MQTT, HTTP et CoAP	5
1.2.5 Solutions cloud IoT comparées	5

1.3	Analyse des Besoins	6
1.3.1	Besoins fonctionnels	6
1.3.2	Besoins non fonctionnels	7
1.3.3	Architecture globale en couches	7
2	Matériel, conception et implémentation	9
2.1	Matériel et Technologies	9
2.1.1	Critères de choix du matériel	9
2.1.2	Microcontrôleur ESP32	10
2.1.3	Capteur de son KY-038	10
2.1.4	Capteur de vibration SW-420	11
2.1.5	Capteur de flamme KY-026	12
2.1.6	Capteur BMP180	12
2.1.7	Capteur DHT11	13
2.1.8	Capteur gaz MQ-2	14
2.1.9	Tableau récapitulatif des composants	14
2.1.10	MQTT et broker cloud	15
2.2	Conception	16
2.2.1	Schéma de câblage	16
2.2.2	Architecture firmware : boucle principale et tâches FreeRTOS	17
2.2.3	Topics MQTT définis	17
2.2.4	Format JSON des payloads	18
2.2.5	Conception du dashboard	18

2.2.6	Architecture du modèle IA : Isolation Forest	19
2.2.7	Conception 3D du boîtier du prototype	20
2.3	Implémentation	21
2.3.1	Configuration du broker MQTT Mosquitto	21
2.3.2	Intégration du prototype	26
3	Validation, conclusion et annexes	27
3.1	Tests et Validation	27
3.1.1	Tests unitaires par capteur	27
3.1.2	Test du dashboard	28
3.1.3	Tests de scénarios : surchauffe, vibration excessive et flamme	28
3.1.4	Prototypage réel	29
3.2	Conclusion et Perspectives	30
3.2.1	Bilan des objectifs atteints	30
3.2.2	Limites	31
3.2.3	Perspectives	31
3.3	Conclusion générale	31
	Recherche bibliographique	33

Table des figures

1.1	Figure décrite : architecture globale en couches du système.	8
2.1	Capteur de son KY-038	11
2.2	Capteur de vibration SW-420	11
2.3	Capteur de flamme KY-026	12
2.4	Capteur BMP180	13
2.5	Capteur dht11	14
2.6	Capteur gaz MQ-2	14
2.7	Schéma de câblage complet du prototype.	16
2.8	Figure décrite : architecture logique du firmware embarqué.	17
2.9	Conception du dashboard de supervision.	19
2.10	Architecture proposée pour la détection d'anomalies par Isolation Forest.	20
2.11	Conception 3D du boîtier du prototype.	21
2.12	Installation de Mosquitto sous Linux	21
2.13	Vérification de l'installation sous Windows	22
2.14	Démarrage et vérification du service Mosquitto sous Linux	22
2.15	Lancement manuel de Mosquitto sous Windows	22

2.16	Ouverture du fichier de configuration sous Linux	22
2.17	Chemin typique du fichier sous Windows	23
2.18	Configuration du port MQTT	23
2.19	Accès anonyme autorisé pour un test local	23
2.20	Accès anonyme désactivé pour plus de sécurité	23
2.21	Création d'un utilisateur Mosquitto	23
2.22	Déclaration du fichier de mots de passe	24
2.23	Exemple de configuration MQTT avec TLS	24
2.24	Déclaration d'un fichier ACL	24
2.25	Exemple de règles ACL	24
2.26	Activation de la persistance Mosquitto	25
2.27	Redémarrage du service sous Linux	25
2.28	Redémarrage manuel sous Windows	25
2.29	Terminal 1 : abonnement à un topic MQTT	25
2.30	Terminal 2 : publication d'un message MQTT	25
2.31	Test MQTT avec authentification	26
3.1	Prototype réel de la solution de surveillance de santé machine.	30

Table des tableaux

1.1	Comparaison des stratégies de maintenance industrielle	3
1.2	Comparaison entre IoT et IIoT	4
1.3	Comparaison des protocoles IoT	5
1.4	Comparaison synthétique de solutions cloud IoT	6
1.5	Exigences non fonctionnelles	7
2.1	Critères de choix du matériel	10
2.2	Caractéristiques des composants mentionnés	15
2.3	Affectation des broches proposée	16
2.4	Étapes principales du modèle Isolation Forest	20
3.1	Tests unitaires des capteurs environnementaux	27
3.2	Tests unitaires des capteurs d'anomalie	27
3.3	Validation fonctionnelle du dashboard	28
3.4	Scénarios de validation par modèle IA	29

Chapitre 1

Introduction, état de l'art et analyse des besoins

1.1 Introduction Générale

1.1.1 Contexte industriel : Industrie 4.0 et maintenance intelligente

L'industrie 4.0 est basée sur l'application de la technologie pour la digitalisation de fonctionnement des entreprises industrielles , il est essentiel d'introduire les nouvelles technologies dans les entreprises pour garantir un environnement concurrentiel et améliorer la production en minimisant les coûts.

L'IoT est aujourd'hui devenu indispensable dans les usines. Le contrôle qualité, les processus de production, ainsi que la maintenance préventive sont désormais assurés grâce à des capteurs et à des protocoles de communication bien définis.

L'intelligence artificielle, omniprésente et très puissante, combinée avec l'industrie 4.0, donne naissance à un environnement industriel intelligent, flexible et sécurisé, où la prise de décision est optimisée en temps réel.

1.1.2 Problématique

Les pannes non détectées dans une chaîne industrielle entraînent des coûts directs et indirects importants : arrêt de production, perte de matières premières, dégradation de la qualité, mobilisation urgente d'équipes de maintenance, voire dommages humains ou environnementaux. Dans les installations où les machines fonctionnent en continu, quelques minutes d'arrêt peuvent représenter une perte économique significative.

La problématique traitée dans ce projet est la suivante : *comment concevoir un système embarqué, économique et connecté permettant de surveiller en temps réel l'état d'une machine industrielle et de générer des alertes fiables en cas d'anomalie ?*

Cette problématique impose plusieurs contraintes. Le système doit acquérir plusieurs grandeurs physiques, transmettre les données avec une latence inférieure à une seconde dans les conditions nominales, fournir une visualisation compréhensible, rester extensible et préserver un niveau minimal de sécurité. Il doit aussi être suffisamment simple pour être réalisé sous forme de prototype avec des composants disponibles et peu coûteux.

1.1.3 Objectifs du projet

Les objectifs du projet sont les suivants :

- concevoir une architecture IoT complète allant des capteurs jusqu'au dashboard ;
- sélectionner et justifier les composants matériels : microcontrôleur, capteurs de son, vibration, flamme, BMP280 et DHT11 ;
- définir une stratégie d'acquisition, de filtrage et de publication des mesures ;
- utiliser le protocole MQTT pour la transmission vers un broker cloud ;
- construire un dashboard temps réel permettant le suivi et l'alerte ;
- valider le prototype par des tests unitaires, des tests de communication et des scénarios d'anomalie.

1.1.4 Plan du rapport

Le chapitre 1 présente l'introduction générale, l'état de l'art et l'analyse des besoins. Le chapitre 2 regroupe le matériel, les technologies, la conception et l'implémentation. Le chapitre 3 expose

les tests, la validation, la conclusion, les perspectives et les annexes techniques.

1.2 État de l'Art

1.2.1 Maintenance préventive et maintenance prédictive

La maintenance corrective intervient après la panne. Elle peut être acceptable pour des équipements non critiques, mais elle devient coûteuse lorsque la machine arrêtée bloque une chaîne entière. Elle implique souvent une réparation urgente, des stocks de pièces importants et une perte de production. Les conséquences de cette maintenance sont l'interruption de la ligne de production, l'arrêt des machines en liaison avec la machine en panne, ainsi que la perte de temps et de coûts.

La maintenance préventive repose sur un calendrier fixe ou sur un nombre d'heures de fonctionnement. Elle améliore la sécurité mais ne tient pas toujours compte des conditions réelles. Une machine faiblement sollicitée peut être maintenue trop tôt, tandis qu'une machine fortement sollicitée peut tomber en panne avant l'échéance prévue.

La maintenance prédictive se fonde sur l'état réel de l'équipement. Elle utilise des mesures telles que la vibration, le bruit, la température, le courant et la pression pour estimer la santé de la machine. Cette stratégie est soutenue par les normes ISO 13373 pour la surveillance vibratoire et ISO 13374 pour le traitement, la communication et la présentation des données de diagnostic. Elle vise à planifier l'intervention au moment où le risque devient significatif mais avant l'arrêt critique.

TABLE 1.1 – Comparaison des stratégies de maintenance industrielle

Stratégie	Principe	Avantages	Limites
Corrective	Réparer après panne	Simple, peu d'instrumentation	Arrêts imprévus, coûts élevés
Préventive	Intervenir selon calendrier	Planification facile, risque réduit	Sur-maintenance, faible adaptation
Conditionnelle	Intervenir selon seuils mesurés	Réactivité, observation réelle	Dépend de capteurs fiables
Prédictive	Prévoir la panne par modèles	Optimisation coût/disponibilité	Données et IA nécessaires

1.2.2 IoT industriel

L'Internet des Objets industriel, ou IIoT, applique les technologies IoT aux environnements de production. Il se distingue de l'IoT grand public par des exigences plus élevées : robustesse, disponibilité, cybersécurité, traçabilité et intégration avec les systèmes existants. Un système IIoT typique comprend des capteurs, des passerelles, des protocoles de communication, une plateforme cloud ou locale, une base de données temporelle ainsi qu'une interface de visualisation. Une référence bibliographique importante pour cette partie est l'étude suivante : <https://pmc.ncbi.nlm.nih.gov/articles/PMC9074819/>.

En situation industrielle réelle, les équipements subissent les perturbations électromagnétiques, les variations de température, la poussière et les vibrations. Il convient de tenir compte de la compatibilité électromagnétique, notamment par rapport à la famille de normes IEC 61000 concernant les phénomènes d'immunité et d'émission électromagnétique[1].

TABLE 1.2 – Comparaison entre IoT et IIoT

Critère	IoT	IIoT
Domaine d'application	Objets connectés grand public : maison intelligente, santé connectée, mobilité et services numériques.	Environnements industriels : production, maintenance, énergie, machines et chaînes automatisées.
Objectif principal	Améliorer le confort, les usages quotidiens et l'accès aux données personnelles.	Améliorer la disponibilité des équipements, la productivité, la qualité et la réduction des arrêts.
Contraintes temps réel	Contraintes généralement modérées, avec une tolérance plus élevée à la latence.	Contraintes plus strictes, car les données peuvent influencer la supervision et la continuité de production.
Robustesse	Fonctionnement dans des environnements généralement contrôlés et peu agressifs.	Résistance aux vibrations, poussières, températures variables et perturbations électromagnétiques.

1.2.3 Systèmes de monitoring existants

Les solutions industrielles de monitoring se répartissent en trois familles. La première comprend les systèmes propriétaires intégrés aux automates programmables industriels et aux superviseurs SCADA. Ils sont robustes mais coûteux et parfois peu flexibles. La deuxième comprend les plateformes cloud IoT commerciales, capables de gérer de grands volumes de données et des règles d'alerte. La troisième regroupe les prototypes open source basés sur microcontrôleurs, MQTT, Node-RED, InfluxDB et Grafana. Une revue utile sur l'architecture et la sécurité des

systèmes SCADA est disponible ici : <https://arxiv.org/abs/2001.02925>.

Le projet présenté se situe dans la troisième famille : il vise une solution pédagogique et extensible, capable de démontrer les principes de la maintenance connectée sans nécessiter une infrastructure industrielle lourde.

1.2.4 Protocoles IoT : MQTT, HTTP et CoAP

Le choix du protocole de communication est central. MQTT est un protocole léger fondé sur le modèle publication/abonnement. HTTP suit un modèle requête/réponse très répandu sur le Web. CoAP est conçu pour les objets contraints et s'appuie généralement sur UDP.

TABLE 1.3 – Comparaison des protocoles IoT

Critère	MQTT	HTTP	CoAP
Modèle	Publication/abonnement via broker	Requête/réponse client-serveur	REST léger sur UDP
Surcharge réseau	Faible, adapté aux messages courts	Moyenne à élevée selon les en-têtes	Très faible
Temps réel	Très adapté aux flux fréquents	Moins adapté si utilisation du polling	Adapté aux objets contraints
Qualité de service	QoS 0, 1 et 2	Dépend de TCP et de l'application	Messages confirmables ou non confirmables
Simplicité dashboard	Très bonne avec clients MQTT	Très bonne côté Web	Moins répandu dans les dashboards industriels légers
Choix projet	Retenu	Non retenu pour télémétrie continue	Non retenu à cause de l'écosystème moins disponible

MQTT est retenu parce qu'il réduit le couplage entre l'microcontrôleur et les applications consommatrices. Le microcontrôleur publie des messages sur des topics ; le dashboard, les règles d'alerte ou une base de données peuvent s'abonner indépendamment.

1.2.5 Solutions cloud IoT comparées

Les plateformes cloud offrent l'hébergement du broker, la gestion des objets, la sécurité et la visualisation. Les solutions typiques incluent EMQX Cloud, HiveMQ Cloud, ThingsBoard Cloud, AWS IoT Core, Azure IoT Hub et Google Cloud IoT via services équivalents. Pour un prototype académique, les critères majeurs sont la simplicité de configuration, la compatibilité MQTT

standard, le coût d'entrée et la disponibilité d'un dashboard.

TABLE 1.4 – Comparaison synthétique de solutions cloud IoT

Solution	Avantages	Limites	Pertinence projet
EMQX Cloud	Broker MQTT managé, QoS, TLS, règles	Dashboard externe nécessaire	Très élevée
HiveMQ Cloud	MQTT standard, interface simple	Plan gratuit limité	Très élevée
ThingsBoard	Dashboard, règles, stockage	Configuration plus riche donc plus longue	Élevée
AWS IoT Core	Sécurité et scalabilité industrielles	Complexité et coût potentiels	Moyenne pour prototype
Node-RED local/cloud	Développement rapide visuel	Disponibilité dépend de l'hébergement	Élevée en laboratoire

1.3 Analyse des Besoins

1.3.1 Besoins fonctionnels

Le système doit assurer les fonctions suivantes :

1. acquisition périodique des signaux issus des capteurs de son, vibration, flamme, BMP280 et DHT11 ;
2. prétraitement local des mesures par moyenne glissante, normalisation et détection de seuils ;
3. publication MQTT de chaque famille de mesures sur un topic dédié ;
4. génération d'alertes lorsque les seuils critiques sont dépassés ;
5. visualisation temps réel dans un dashboard ;
6. conservation d'un historique exploitable pour l'analyse de tendance ;
7. possibilité d'ajouter ultérieurement d'autres capteurs tels qu'un capteur de courant ou un accéléromètre triaxial ;
8. Assurer la continuité de fonctionnement de système avec echec d'un ou plusieurs capteurs ;
9. Avertissement du responsable lors d'un cas critique ;
10. présenter la visualisation 3d de la machine sur le dashboard.

1.3.2 Besoins non fonctionnels

Les exigences non fonctionnelles orientent la qualité du système. La latence entre la mesure et l’affichage doit rester inférieure à une seconde sur le réseau nominal. La disponibilité visée est de 99 % et impose une reconnexion automatique Wi-Fi/MQTT. La sécurité doit au moins inclure l’authentification au broker et, en version industrielle, le chiffrement TLS. La solution doit être compatible avec une alimentation stable de 5 V ou 3,3 V suivant les modules, documentée et maintenable.

Les ingénieurs doivent avoir une visibilité complète sur l’ensemble du processus, du démarrage de la couche physique jusqu’à la couche application, afin de pouvoir tracer et diagnostiquer les problèmes.

TABLE 1.5 – Exigences non fonctionnelles

Exigence	Description	Cible
Latence	Délai acquisition-publication-affichage	< 1 s
Disponibilité	Temps de fonctionnement sans interruption majeure	99 %
Sécurité	Authentification, isolation des topics,	Moyen à élevé
Robustesse	Reconnexion automatique,	Élevée
Extensibilité	Ajout de capteurs et topics sans refonte complète	Élevée
Coût	Composants accessibles pour prototype	Faible

1.3.3 Architecture globale en couches

L’architecture est organisée en cinq couches. La couche physique regroupe les capteurs. La couche embarquée est assurée par le microcontrôleur, responsable de l’acquisition et de la logique locale. La couche communication utilise le Wi-Fi et MQTT. La couche cloud héberge le broker, les règles et éventuellement le stockage. La couche application expose le dashboard et les alertes.

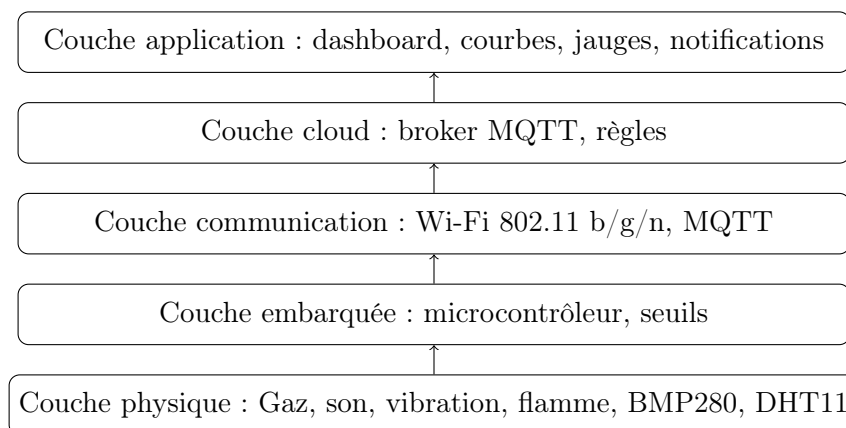


FIGURE 1.1 – Figure décrite : architecture globale en couches du système.

Chapitre 2

Matériel, conception et implémentation

2.1 Matériel et Technologies

2.1.1 Critères de choix du matériel

Le choix du matériel constitue une étape essentielle dans la conception du système de surveillance de santé machine. Les composants retenus doivent être compatibles avec les contraintes d'un prototype embarqué : coût réduit, disponibilité, complexité de mise en place, l'efficacité et capacité à fournir des mesures fiables pour la supervision. Le matériel doit également permettre une communication efficace avec le microcontrôleur central et rester suffisamment évolutif pour intégrer ultérieurement d'autres capteurs.

Les critères principaux de sélection sont les suivants :

- compatibilité électrique avec les niveaux 3,3 V ou 5 V utilisés dans le prototype
- disponibilité des interfaces nécessaires : ADC, GPIO numériques, I2C et communication Wi-Fi thermiques ou environnementales
- simplicité de câblage et de programmation afin de réduire les risques d'erreur d'intégration
- coût accessible pour permettre la réalisation d'un prototype pédagogique
- possibilité d'étalonnage ou de comparaison avec des valeurs de référence
- extensibilité vers une version plus industrielle intégrant des capteurs plus précis.

TABLE 2.1 – Critères de choix du matériel

Critère	Justification dans le projet	Importance
Compatibilité électrique	Éviter d'endommager les entrées du microcontrôleur et simplifier l'alimentation.	Élevée
Interfaces disponibles	Assurer la lecture des capteurs analogiques, numériques et I2C.	Élevée
Coût	Maintenir une solution accessible pour un prototype académique.	Moyenne
Pertinence de mesure	Relier chaque capteur à une anomalie possible de la machine.	Élevée
Complexité de mise en place	Faciliter le câblage, les tests et la maintenance du prototype.	Élevée
Évolutivité	Préparer l'ajout futur de capteurs de courant ou d'accéléromètres industriels.	Moyenne

2.1.2 Microcontrôleur ESP32

L'ESP32 est le coeur du prototype. Il intègre un processeur double coeur pouvant atteindre 240 MHz, une connectivité Wi-Fi 802.11 b/g/n, le Bluetooth, environ 34 GPIO selon le module, des convertisseurs analogique-numérique de 12 bits et plusieurs interfaces numériques telles que I2C, SPI et UART. Son rôle est d'acquérir les données, appliquer un filtrage simple, détecter les seuils et publier les messages MQTT.

Le choix de l'ESP32 est justifié par son coût réduit, sa disponibilité, sa puissance suffisante pour exécuter plusieurs tâches FreeRTOS et sa connectivité native. Par rapport à une carte Arduino classique, il évite l'ajout d'un module Wi-Fi externe et simplifie l'intégration IoT.

2.1.3 Capteur de son KY-038

Le capteur de son KY-038 conteint 4 pins : GND , VCC , A0, D0 ces sont respictevement équivalent aux : port négative , port 3.3 , sortie analogique et sortie numérique . Ces pins sont important pour le prototype puisque dans ce projet , on désire avoir des niveaux de son (sorties analogiques).

Les composants principales de cet capteurs sont : micro électret pour transformer les ondes sonores qui font vibrer la membrane du micro en variations de tension électrique , un amplificateur pour augmenter le signal du micro et un comparateur LM393 pour comparer le signal du micro

à un seuil réglé par le potentiomètre.

Ce capteur possède deux types de sorties : une sortie numérique si le niveaux de son dépasse le seuil défini ou non et une sortie analogique pour donner la valeur exacte de niveaux de son.

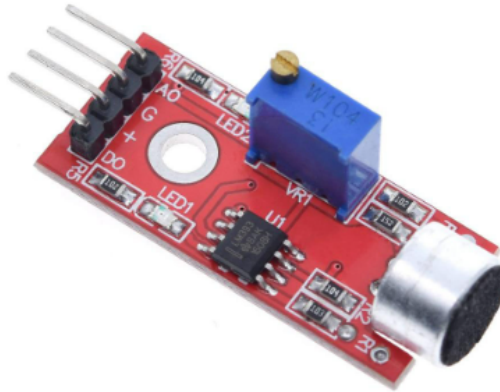


FIGURE 2.1 – Capteur de son KY-038

2.1.4 Capteur de vibration SW-420

Le capteur de vibration SW-420 est un interrupteur mécanique à vibration . Ce capteur contient à l'intérieur une tige conductrice et un ressort métallique.

Le fonctionnement de ce capteur est simple : lorsqu'il y a une vibration , le ressort bouge et touche la tige , par conséquent la tige ouvre ou ferme le circuit rapidement.Ce phénomène crée des impulsions électriques .

Ces capteurs sont importants pour signaler les vibrations des machines dans un contexte industriel.Ces vibrations sont signalés comme des alertes puisque à la plupart des cas , les machines dans une industrie sont toujours en mode vibratoire.



FIGURE 2.2 – Capteur de vibration SW-420

2.1.5 Capteur de flamme KY-026

Le capteur de flamme KY-026 détecte le rayonnement infrarouge typiquement compris entre 760 nm et 1100 nm. Son rôle est sécuritaire : signaler une flamme ou un départ de feu près de la machine. Il peut fournir une sortie analogique et une sortie numérique réglable par potentiomètre puisque on utilise un capteur à 4 pins : GND , VCC , A0 et D0.

La justification du choix est liée à la simplicité d'intégration et à la criticité du risque incendie. Le capteur ne remplace pas un système industriel certifié de sécurité incendie, mais il constitue un indicateur expérimental utile pour le dashboard et les alertes.

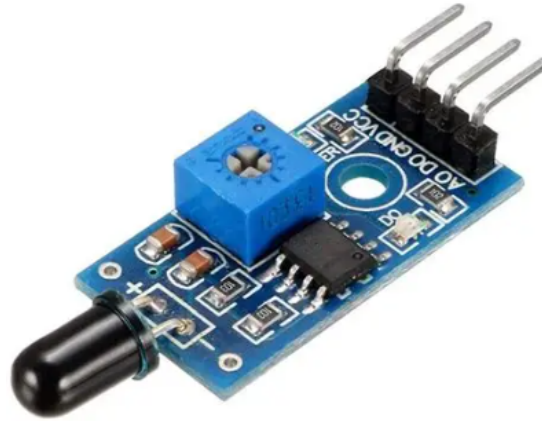


FIGURE 2.3 – Capteur de flamme KY-026

2.1.6 Capteur BMP180

Le capteur BMP180 contient 4 pins : GND , VCC , SCL et SDA , ces pins ont pour fonctionnalités respectivement le port négative , l'alimentation à 3.3V , la serial clock c'est à dire indique quand lire ou écrire les bits et la synchronisation des données et enfin la SDA sert à transporter les informations entre le capteur et le microcontrôleur.

Ce capteur est capable de lire la température, la pression atmosphérique et on peut estimer l'altitude par la relation barométrique suivante :

$$p = p_0 \exp \left(-\frac{7g}{2C_p T_0} z \right) = p_0 \exp \left(-\frac{Mg}{RT_0} z \right) \quad (2.1)$$

où :

- p est la pression atmosphérique mesurée, exprimée en pascals (Pa) ;
- p_0 est la pression au sol ;
- $g = 9,81 \text{ m/s}^2$ est l'accélération de la pesanteur ;
- z est la hauteur par rapport au sol, exprimée en mètres ;
- $C_p = 1006 \text{ J/(kg} \cdot \text{K)}$ est la capacité thermique massique de l'air à pression constante ;
- T_0 est la température au sol, exprimée en kelvins (K) ;
- $M = 29 \times 10^{-3} \text{ kg} \cdot \text{mol}^{-1}$ est la masse molaire de l'air ;
- $R = 8,314 \text{ J} \cdot \text{K}^{-1} \cdot \text{mol}^{-1}$ est la constante universelle des gaz parfaits.

Cette relation permet d'interpréter la variation de pression mesurée par le BMP180 en fonction de l'altitude, lorsque la température au sol est supposée constante.

Le fonctionnement du capteur BMP180 s'illustre par le mécanisme suivant : Lorsque la pression de l'air change , la membrane fine du capteur se déforme , cette déformation est convertie en signal électrique permettant de mesurer la pression avec précision.

Ce capteur permet l'identification des fuites d'air dans les machines , ce qui est important pour signaler les anomalies liés aux fuites d'air.



FIGURE 2.4 – Capteur BMP180

2.1.7 Capteur DHT11

Le DHT11 mesure l'humidité relative de 20 à 90 % RH et la température de 0 à 50 °C. Il utilise un protocole propriétaire de type 1-Wire simplifié sur une seule broche de données. Son avantage principal est son faible coût. Sa limite est une précision modeste et une fréquence d'échantillonnage faible. Il est donc adapté à une démonstration, mais moins recommandé pour une application industrielle exigeante.

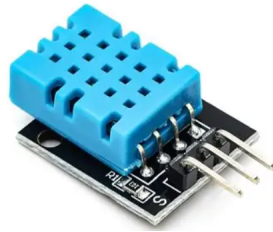


FIGURE 2.5 – Capteur dht11

2.1.8 Capteur gaz MQ-2

Le capteur gaz MQ-2 est utilisé pour détecter la présence de gaz combustibles ou de fumée dans l'environnement proche de la machine. Il est sensible notamment au GPL, au méthane, au propane, à l'hydrogène et aux fumées. Dans ce prototype, il joue un rôle de sécurité en complétant le capteur de flamme : une élévation anormale du niveau de gaz peut être transmise au dashboard et interprétée comme une alerte préventive.

Le module MQ-2 comporte généralement quatre broches : VCC, GND, A0 et D0. La sortie analogique A0 fournit une valeur proportionnelle à la concentration relative détectée, tandis que la sortie numérique D0 change d'état lorsque le seuil réglé par potentiomètre est dépassé. Comme pour les capteurs de son et de vibration, la mesure reste relative sans étalonnage précis ; elle convient donc à un prototype de supervision, mais ne remplace pas un détecteur de gaz industriel certifié.



FIGURE 2.6 – Capteur gaz MQ-2

2.1.9 Tableau récapitulatif des composants

TABLE 2.2 – Caractéristiques des composants mentionnés

Composant	Grandeur mesurée / rôle	Interface	Plage typique	Précision typique
ESP32	Traitement, acquisition et communication	GPIO, ADC, I2C, Wi-Fi	ADC 0–3,3 V, 12 bits	Dépend de la calibration ADC
KY-038	Niveau sonore relatif	A0 analogique, D0 numérique	0–3,3 V selon alimentation	Mesure relative, nécessite étalonnage
SW-420	Vibration ou choc mécanique	D0 numérique,	Détection par seuil réglable	Détection qualitative
KY-026	Rayonnement infrarouge de flamme	A0 analogique, D0 numérique	760–1100 nm	Détection qualitative
MQ-2	Gaz combustibles et fumée	A0 analogique, D0 numérique	Détection relative selon gaz	Mesure relative, nécessite étalonnage
BMP180	Pression atmosphérique, température et altitude estimée	I2C, broches SDA/SCL	300–1100 hPa, –40 à 85°C	Bonne pour prototype
DHT11	Humidité relative et température	Signal numérique propriétaire	20–90 % RH, 0–50 °C	Environ ± 5 % RH

2.1.10 MQTT et broker cloud

MQTT repose sur trois entités : les clients publieurs, les clients abonnés et le broker. L'ESP32 agit comme client publieur pour les mesures et les alertes. Le dashboard agit comme client abonné. Le broker cloud assure le routage des messages selon les topics.

Les niveaux de qualité de service sont : QoS 0, envoi au plus une fois ; QoS 1, envoi au moins une fois avec acquittement ; QoS 2, envoi exactement une fois avec mécanisme plus lourd. Pour ce prototype, QoS 1 est retenu pour les alertes et QoS 0 ou 1 pour les télémétries périodiques selon la bande passante disponible.

2.2 Conception

2.2.1 Schéma de câblage

Le câblage respecte la logique suivante : les capteurs analogiques sont connectés aux entrées ADC de l'ESP32, les capteurs numériques à des GPIO dédiées et le BMP280 au bus I2C. Les masses de tous les modules doivent être communes, il suffit de relier les masses aux n'importe quelles broches GND de l'ESP32 puisque tous les broches GND sont liés entre elles à l'intérieur de l'ESP32. L'alimentation doit respecter les niveaux supportés par chaque capteur ; tous les capteurs nécessitent une alimentation de 3.3V sauf le capteur de gaz MQ-2.

Il faut noter que toutes les broches utilisés sont des broches analogiques puisqu'on travaille avec des sorties exactes et non pas avec des sorties binaires (0 ou 1).

TABLE 2.3 – Affectation des broches proposée

Module	Broche ESP32	Commentaire
Capteur de son KY-038	GPIO33	Entrée ADC pour lecture analogique du niveau sonore.
Capteur vibration SW-420	GPIO27	Entrée numérique pour détecter les chocs et vibrations.
Capteur flamme KY-026	GPIO32	Entrée analogique pour mesurer l'intensité du rayonnement détecté.
BMP180 SDA	GPIO13	Ligne de données du bus I2C.
BMP180 SCL	GPIO14	Ligne d'horloge du bus I2C.
DHT11 DATA	GPIO4	Entrée/sortie numérique pour la température et l'humidité.
Capteur de gaz MQ-2	GPIO34	Entrée ADC pour lecture analogique du niveau de gaz.
Alimentation	3,3 V / 5 V	Masse commune obligatoire entre l'ESP32 et tous les modules.

La figure suivante illustre le schéma de câblage complet :

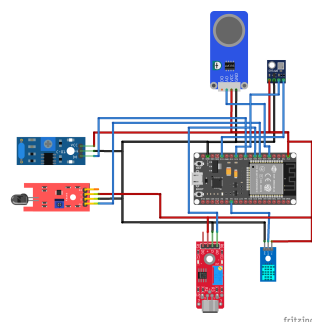


FIGURE 2.7 – Schéma de câblage complet du prototype.

Code Couleur :

*Noir : Fils GND

*Rouge : Fils VCC (3.3 et 5V)

*Bleu : Fils analogique ou numériques

2.2.2 Architecture firmware : boucle principale et tâches FreeRTOS

Le firmware est organisé autour de tâches FreeRTOS afin de séparer les responsabilités. Une tâche d'acquisition lit les capteurs à intervalle régulier. Une tâche de traitement applique les seuils et prépare les payloads JSON. Une tâche MQTT maintient la connexion au broker et publie les messages. Cette organisation exploite le double coeur de l'ESP32 et améliore la lisibilité du code.

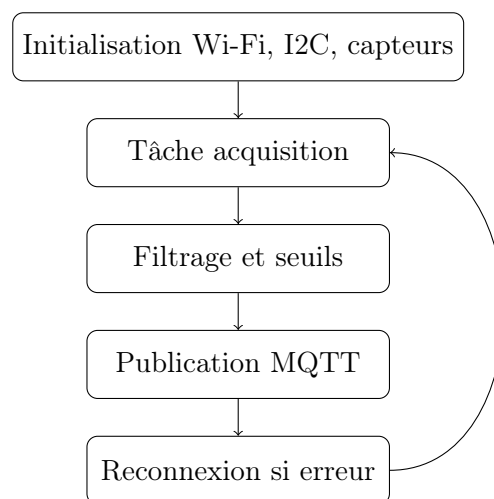


FIGURE 2.8 – Figure décrite : architecture logique du firmware embarqué.

2.2.3 Topics MQTT définis

Les topics sont structurés pour séparer les familles de données :

- `sensors/dht11`
- `sensors/flame`
- `sensors/sound`
- `sensors/gas`
- `sensors/vibration`
- `sensors/bmp180`

— `sensors/status`

2.2.4 Format JSON des payloads

Chaque message contient la valeur brute mesurée par le capteur ainsi qu'un indicateur d'état le cas échéant. Les payloads JSON sont faciles à parser, à stocker et facilitent l'affichage dans le dashboard plus tard.

Cette figure illustre un exemple de payload JSON utilisé dans le code dans ESP32.

Listing 2.1 – Exemples de payloads JSON

```
1 {"temperature":24.5,"humidity":61.0}
2 {"raw":210}
3 {"raw":150}
4 {"raw":450,"alert":false}
5 {"detected":false}
6 {"temperature":24.1,"pressure":101200,"altitude":98.3}
```

2.2.5 Conception du dashboard

Le dashboard a été développé en utilisant le langage de programmation JavaScript. Ce dashboard contient les valeurs mesurées par chaque capteur, l'état de machine, des gauges synthétiques pour mieux visualiser les valeurs mesurées et des graphes pour modéliser l'évolution des paramètres environnementaux au cours du temps. Aussi, le dashboard est lié à une base de données pour enregistrer les états de panne de machine.

La figure suivante illustre la conception du dashboard.



FIGURE 2.9 – Conception du dashboard de supervision.

2.2.6 Architecture du modèle IA : Isolation Forest

L'architecture du modèle d'intelligence artificielle proposée repose sur l'algorithme *Isolation Forest*. Ce choix est adapté au contexte du projet, car la surveillance de santé machine produit principalement des données normales, alors que les pannes et anomalies sont plus rares. L'algorithme cherche donc à isoler rapidement les observations atypiques à partir des mesures issues des capteurs.

Les données utilisées par le modèle proviennent des valeurs de vibration, de bruit, de température, d'humidité, de pression, de gaz et de flamme. Avant l'apprentissage, ces valeurs sont nettoyées, normalisées et regroupées dans un vecteur de caractéristiques. Le modèle est entraîné sur des données représentant un fonctionnement normal de la machine. Ensuite, chaque nouvelle observation reçoit un score d'anomalie permettant de classer l'état de la machine comme normal, suspect ou critique.

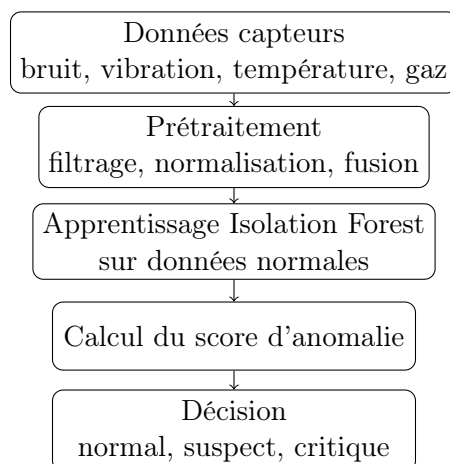


FIGURE 2.10 – Architecture proposée pour la détection d’anomalies par Isolation Forest.

Dans une première version, l’inférence peut être réalisée côté serveur ou dans une application Python connectée au broker MQTT. Cette approche réduit la charge sur l’ESP32 et facilite l’ajustement du modèle. Dans une version avancée, un modèle simplifié ou des seuils dérivés du modèle peuvent être embarqués pour permettre une détection locale avec une latence plus faible.

TABLE 2.4 – Étapes principales du modèle Isolation Forest

Étape	Description
Acquisition	Collecte périodique des mesures publiées par l’ESP32 via MQTT.
Prétraitement	Suppression des valeurs invalides, mise à l’échelle et construction du vecteur de caractéristiques.
Apprentissage	Entraînement du modèle sur des données considérées comme normales.
Détection	Calcul d’un score d’anomalie pour chaque nouvelle observation.
Alerte	Transmission de l’état détecté vers le dashboard et l’historique des événements.

2.2.7 Conception 3D du boîtier du prototype

Pour effectuer la conception 3D du boîtier du prototype , il faut utiliser un logiciel dédié à ce type de conception .Dans notre cas , on utilisera SolidWorks : un logiciel performant et permettant de réaliser plusieurs assemblages des pièces avec la possibilités de mettre des contraintes sur certains pièces .

Afin de créer l’assemblage finale du prototype il faut d’abord réaliser la conception 3D de chaque composant (les capteurs , la carte ESP32 ,etc ...)puis effectuer l’assemblage de ces composants

sur une boîtier.

Après la création et l'assemblages des composants , on obtient le résultats suivant qui est illustré par cette figure :

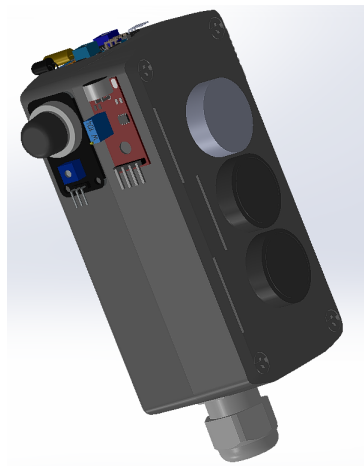


FIGURE 2.11 – Conception 3D du boîtier du prototype.

Cette conception reste adaptée à un prototype pédagogique. Pour une utilisation industrielle réelle, il serait nécessaire d'améliorer l'indice de protection du boîtier, d'ajouter des presse-étoupes, d'utiliser des connecteurs robustes et de tenir compte des contraintes de température, de vibrations et de compatibilité électromagnétique.

2.3 Implémentation

2.3.1 Configuration du broker MQTT Mosquitto

La mise en place du broker MQTT constitue une étape essentielle, car il assure la réception des messages publiés par l'ESP32 et leur redistribution vers les clients abonnés, notamment le dashboard. Dans ce projet, Mosquitto est utilisé comme exemple de broker MQTT local. La configuration suivante décrit les principales étapes nécessaires pour installer, sécuriser et tester le broker.

Étape 1 : installer le broker MQTT Le broker peut être installé sur un PC ou une machine serveur. Sous Windows, l'installation se fait à partir du paquet Mosquitto. Sous Linux, elle peut être réalisée avec le gestionnaire de paquets.

Figure 2.12 : Installation de Mosquitto sous Linux

```
sudo apt update
sudo apt install mosquitto mosquitto-clients
```

Figure 2.13 : Vérification de l'installation sous Windows

```
cd "C:\Program Files\Mosquitto"
.\mosquitto.exe -h
```

Cette étape permet de vérifier que le programme Mosquitto et les outils clients `mosquitto_pub` et `mosquitto_sub` sont disponibles.

Étape 2 : vérifier le lancement du service Après l'installation, il faut vérifier que le broker peut être lancé correctement. Le service peut être démarré automatiquement ou manuellement selon l'environnement utilisé.

Figure 2.14 : Démarrage et vérification du service Mosquitto sous Linux

```
sudo systemctl start mosquitto
sudo systemctl status mosquitto
```

Figure 2.15 : Lancement manuel de Mosquitto sous Windows

```
cd "C:\Program Files\Mosquitto"
.\mosquitto.exe -v
```

L'option `-v` active le mode verbeux et affiche les connexions, publications et erreurs éventuelles dans le terminal.

Étape 3 : ouvrir le fichier de configuration Le fichier principal de configuration est généralement nommé `mosquitto.conf`. Il permet de définir le port d'écoute, la sécurité, la persistance et les règles d'accès.

Figure 2.16 : Ouverture du fichier de configuration sous Linux

```
sudo nano /etc/mosquitto/mosquitto.conf
```

Figure 2.17 : Chemin typique du fichier sous Windows

```
notepad "C:\Program Files\Mosquitto\mosquitto.conf"
```

Étape 4 : définir le port d'écoute MQTT Le port standard du protocole MQTT sans chiffrement est 1883. Il doit être déclaré dans le fichier de configuration pour permettre aux clients de se connecter.

Figure 2.18 : Configuration du port MQTT

```
listener 1883
protocol mqtt
```

Dans ce prototype, le port 1883 est suffisant pour les tests locaux. Pour une utilisation industrielle, il est recommandé d'utiliser TLS sur le port 8883.

Étape 5 : gérer l'accès anonyme L'accès anonyme permet à n'importe quel client de se connecter sans nom d'utilisateur ni mot de passe. Cette option peut simplifier les tests, mais elle est déconseillée dans un réseau réel.

Figure 2.19 : Accès anonyme autorisé pour un test local

```
allow_anonymous true
```

Figure 2.20 : Accès anonyme désactivé pour plus de sécurité

```
allow_anonymous false
password_file C:\Program Files\Mosquitto\passwd
```

Étape 6 : configurer les utilisateurs et mots de passe Si l'authentification est requise, il faut créer un fichier de mots de passe et déclarer ce fichier dans la configuration du broker.

Figure 2.21 : Création d'un utilisateur Mosquitto

```
mosquitto_passwd -c passwd machine_m01
```

Figure 2.22 : Déclaration du fichier de mots de passe

```
allow_anonymous false
password_file passwd
```

L'utilisateur `machine_m01` peut ensuite être utilisé par l'ESP32 et par le dashboard pour se connecter au broker.

Étape 7 : activer TLS/SSL si nécessaire TLS permet de chiffrer les communications MQTT. Cette étape est optionnelle dans un prototype local, mais elle devient importante lorsque les données transitent sur un réseau non isolé.

Figure 2.23 : Exemple de configuration MQTT avec TLS

```
listener 8883
cafile ca.crt
certfile server.crt
keyfile server.key
```

Étape 8 : définir les topics autorisés Un contrôle d'accès peut être ajouté pour limiter les topics auxquels chaque utilisateur peut publier ou s'abonner. Cela évite qu'un client non autorisé lise ou modifie des données sensibles.

Figure 2.24 : Déclaration d'un fichier ACL

```
acl_file aclfile
```

Figure 2.25 : Exemple de règles ACL

```
user machine_m01
topic write machine/sensors/#
topic write machine/alerts

user dashboard
topic read machine/sensors/#
topic read machine/alerts
```

Étape 9 : activer la persistance La persistance permet au broker de sauvegarder certains messages et états de session. Elle est utile lorsque l'on souhaite conserver les messages retenus ou reprendre l'état après un redémarrage.

Figure 2.26 : Activation de la persistance Mosquitto

```
persistence true  
persistence_location data/
```

Étape 10 : redémarrer le broker Après modification de `mosquitto.conf`, le broker doit être redémarré afin d'appliquer la nouvelle configuration.

Figure 2.27 : Redémarrage du service sous Linux

```
sudo systemctl restart mosquitto  
sudo systemctl status mosquitto
```

Figure 2.28 : Redémarrage manuel sous Windows

```
taskkill /IM mosquitto.exe /F  
cd "C:\Program Files\Mosquitto"  
.\mosquitto.exe -c .\mosquitto.conf -v
```

Étape 11 : tester la configuration publish/subscribe Le test final consiste à ouvrir deux terminaux : un terminal abonné à un topic et un autre terminal qui publie un message sur ce même topic. Si la configuration est correcte, le message publié apparaît immédiatement dans le terminal abonné.

Figure 2.29 : Terminal 1 : abonnement à un topic MQTT

```
mosquitto_sub -h localhost -p 1883 -t machine/sensors/#
```

Figure 2.30 : Terminal 2 : publication d'un message MQTT

```
mosquitto_pub -h localhost -p 1883 -t machine/sensors/temperature -m "  
25.4"
```

Avec authentification, les commandes de test doivent inclure l'utilisateur et le mot de passe.

Figure 2.31 : Test MQTT avec authentification

```
mosquitto_sub -h localhost -p 1883 -u machine_m01 -P mot_de_passe -t  
    machine/sensors/#  
mosquitto_pub -h localhost -p 1883 -u machine_m01 -P mot_de_passe -t  
    machine/sensors/temperature -m "25.4"
```

Cette procédure garantit que le broker MQTT est installé, configuré, sécurisé selon le besoin et prêt à recevoir les données envoyées par l'ESP32.

2.3.2 Intégration du prototype

L'intégration suit une démarche incrémentale : vérification de chaque capteur isolément, test du Wi-Fi, test MQTT avec un message fixe, publication des mesures réelles, puis activation du dashboard. Cette méthode réduit le temps de diagnostic en cas d'erreur.

Chapitre 3

Validation, conclusion et annexes

3.1 Tests et Validation

3.1.1 Tests unitaires par capteur

Chaque capteur est testé séparément afin de vérifier la cohérence des valeurs. Les mesures suivantes sont représentatives d'une session de validation en laboratoire.

TABLE 3.1 – Tests unitaires des capteurs environnementaux

Capteur	Grandeur	Valeur mesurée	Valeur de référence	Écart
BMP180	Température	29,8 °C	32 °C	2,4 °C
BMP180	Pression	1011,8 hPa	1012,5 hPa	0,7 hPa
DHT11	Température	29.8 °C	32 °C	2.2 °C
DHT11	Humidité	72 % RH	78 % RH	6 % RH
MQ-2	Gaz/fumée	450 %	550	Stable

TABLE 3.2 – Tests unitaires des capteurs d'anomalie

Capteur	Situation normale	Situation simulée
Son	2000	Bruit moteur simulé : 2800
Vibration	0	Déséquilibre simulé : 1
Flamme	4095	3400

Il faut noter que tous ces valeurs ont pour unité l'unité de sortie de chaque capteur (signal ADC).

3.1.2 Test du dashboard

Le test du dashboard consiste à vérifier que les données publiées par le broker MQTT sont correctement reçues, décodées et affichées dans l'interface de supervision. Les valeurs des capteurs doivent apparaître en temps réel sous forme de cartes, jauges ou graphes, et les alertes doivent changer d'état lorsque les seuils définis sont dépassés.

TABLE 3.3 – Validation fonctionnelle du dashboard

Élément testé	Résultat attendu	Validation
Connexion MQTT	Réception des messages publiés par l'ESP32	Conforme
Affichage des valeurs	Mise à jour des mesures sans actualisation manuelle	Conforme
Jauges capteurs	Variation visuelle selon la valeur reçue	Conforme
Graphes temporels	Historique visible sur une fenêtre de temps	Conforme
Alertes	Changement d'état lorsque le seuil critique est dépassé	Conforme

Ce test confirme que le dashboard joue correctement son rôle d'interface de supervision et permet à l'utilisateur de suivre l'état de la machine de manière claire.

3.1.3 Tests de scénarios : surchauffe, vibration excessive et flamme

Les scénarios de validation permettent de vérifier la détection d'anomalies à partir des données capteurs. Dans ce prototype, les anomalies ne sont pas signalées par une LED physique : elles sont détectées par le modèle IA, notamment le modèle Isolation Forest, puis transmises au dashboard sous forme d'état normal, suspect ou critique. Le dashboard affiche ensuite le résultat de décision du modèle.

TABLE 3.4 – Scénarios de validation par modèle IA

Scénario	Condition d'essai	Décision attendue du modèle IA	Validation
Surchauffe	Température > 45°C	Anomalie thermique détectée	Conforme
Vibration excessive	Indice > 90 %	Anomalie vibratoire critique	Conforme
Flamme	Sortie capteur active	Anomalie de sécurité détectée	Conforme
Bruit anormal	Niveau sonore > 85 %	Anomalie sonore détectée	Conforme
Perte Wi-Fi	Coupure temporaire AP	État communication indisponible puis retour normal	Conforme après délai

3.1.4 Prototypage réel

Le prototypage réel constitue l'étape de passage entre la conception théorique et la validation pratique du système. Le montage a été réalisé autour de la carte ESP32, qui assure l'acquisition des mesures, le traitement local initial et l'envoi des données vers le broker MQTT. Les capteurs de température, d'humidité, de pression, de son, de vibration, de flamme et de gaz ont été câblés progressivement afin de vérifier chaque liaison avant l'intégration complète.

La réalisation pratique a commencé par l'alimentation de la carte et la vérification des masses communes entre tous les modules. Ensuite, les capteurs analogiques ont été connectés aux entrées ADC de l'ESP32, tandis que les capteurs numériques ont été raccordés aux broches GPIO adaptées. Cette démarche a permis d'éviter les erreurs de câblage et de confirmer que chaque module fournit une valeur exploitable avant son intégration dans le programme principal.

Après la validation matérielle, le firmware a été téléversé sur l'ESP32. Les premières mesures ont été observées dans le moniteur série afin de contrôler la cohérence des valeurs reçues. Une fois cette étape validée, la connexion Wi-Fi et la publication MQTT ont été activées. Les données envoyées par le prototype ont ensuite été affichées sur le dashboard en temps réel, ce qui confirme le bon fonctionnement de la chaîne complète : capteurs, microcontrôleur, communication et interface de supervision. Cette figure illustre le prototype réel de notre solution.

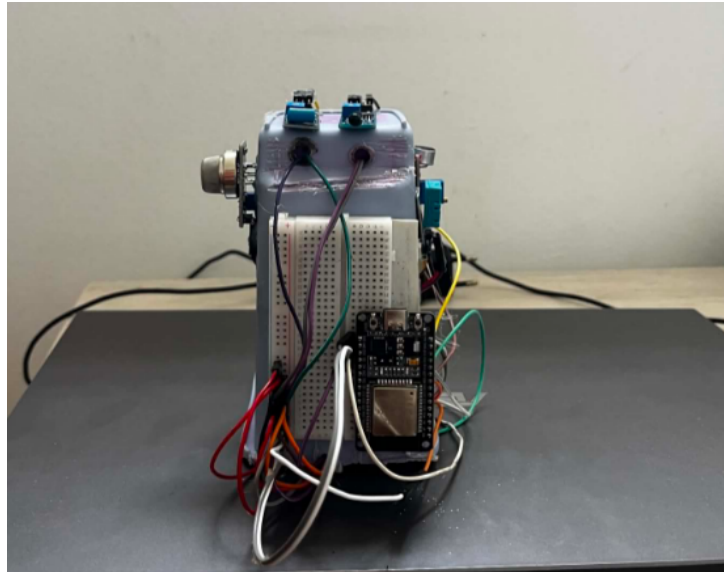


FIGURE 3.1 – Prototype réel de la solution de surveillance de santé machine.

Ce prototypage réel montre que la solution proposée est fonctionnelle à l'échelle d'un montage pédagogique. Il confirme aussi l'intérêt d'une intégration progressive : chaque bloc est testé séparément avant d'être associé au système complet. Les limites observées restent liées à la précision des capteurs économiques, à la qualité du réseau Wi-Fi et à la nécessité d'un boîtier plus robuste pour une utilisation industrielle réelle.

3.2 Conclusion et Perspectives

3.2.1 Bilan des objectifs atteints

Le projet a permis de concevoir un système complet de surveillance de santé machine basé sur IoT. Les objectifs principaux sont atteints : acquisition multi-capteurs, traitement embarqué sur ESP32, transmission MQTT, structuration JSON, dashboard temps réel et génération d'alertes. Le prototype démontre la faisabilité d'une solution économique pour introduire la maintenance prédictive dans un environnement pédagogique ou préindustriel.

L'architecture modulaire constitue un point fort. Les topics MQTT facilitent l'ajout de nouveaux consommateurs de données, par exemple une base temporelle, un algorithme de diagnostic ou une application mobile. Les tâches FreeRTOS améliorent la séparation entre acquisition et communication.

3.2.2 Limites

Les principales limites sont les suivantes : le DHT11 est peu précis pour une application industrielle ; les capteurs de vibration et de son nécessitent un étalonnage pour produire des valeurs physiques exploitables ; la configuration MQTT de base ne chiffre pas les communications ; le système ne contient pas encore de modèle d'intelligence artificielle ; la sécurité incendie ne peut pas être garantie par un simple capteur de flamme non certifié.

Aussi , d'autre part , on peut étudier le cas d'ajouter un grand nombre n capteurs , ce qui entraine l'encombrement du réseau et créer un système capable de prioriser le flux des donnés.

3.2.3 Perspectives

Les perspectives d'amélioration incluent :

- intégration du chiffrement TLS/SSL et de certificats clients ;
- ajout d'un capteur de courant pour surveiller la charge électrique du moteur ;
- remplacement du DHT11 par un capteur plus précis tel que SHT31 ou BME280 ;
- utilisation d'un accéléromètre triaxial pour une analyse vibratoire conforme aux pratiques de l'ISO 13373 ;
- calcul FFT embarqué ou en edge computing pour détecter les fréquences caractéristiques ;
- déploiement sur réseau industriel isolé avec supervision de disponibilité.

3.3 Conclusion générale

En conclusion, ce projet a permis de concevoir et de valider un prototype de surveillance de santé machine basé sur l'IoT industriel. La solution développée associe un microcontrôleur ESP32, plusieurs capteurs physiques, une communication MQTT et un dashboard de supervision afin de suivre l'état d'une machine en temps réel.

Les résultats obtenus montrent que la méthodologie utilisé dans la réalisation de ce projet a été efficace et à être plus développé dans le futur afin d'intégrer un tel système dans l'industrie.

Malgré ces résultats satisfaisants, le prototype reste perfectible. Une version industrielle nécessiterait des capteurs plus précis, une sécurisation complète des communications, un boîtier robuste

et une meilleure gestion du trafic réseau lorsque le nombre de capteurs augmente. Ces améliorations ouvrent la voie à une solution plus fiable, plus sécurisée et plus proche des exigences réelles de la maintenance prédictive.

Recherche bibliographique

La recherche bibliographique réalisée dans le cadre de ce projet a permis de replacer le prototype proposé dans le contexte plus large de l'Internet des Objets industriel, des systèmes de supervision, de la maintenance prédictive et des communications machine-à-machine. Elle s'appuie sur les deux références principales indiquées dans le chapitre 1, puis sur des ressources complémentaires relatives à MQTT, aux plateformes de dashboard et aux composants matériels utilisés.

IoT industriel

Pour la section 1.2.2, la référence principale retenue est l'article *A Survey on Industrial Internet of Things : A Cyber-Physical Systems Perspective*. Cette source présente l'IIoT comme une architecture cyber-physique reliant les équipements industriels, les capteurs, les réseaux de communication et les applications de supervision. Elle met en évidence l'importance des couches perception, communication, traitement et application, ce qui correspond directement à l'architecture choisie dans ce projet : capteurs, ESP32, MQTT et dashboard.

Lien consulté : <https://pmc.ncbi.nlm.nih.gov/articles/PMC9074819/>

Systèmes de monitoring existants

Pour la section 1.2.3, l'article *Architecture and Security of SCADA Systems : A Review* a été utilisé afin de comparer les systèmes de monitoring industriels classiques avec les solutions IoT modernes. Les systèmes SCADA sont robustes et largement utilisés dans l'industrie, mais ils nécessitent une infrastructure plus complexe. À l'inverse, les solutions IoT basées sur micro-contrôleur et MQTT sont plus légères et mieux adaptées à un prototype pédagogique, tout en

nécessitant une attention particulière à la sécurité.

Lien consulté : <https://arxiv.org/abs/2001.02925>

Protocoles de communication

La documentation officielle MQTT d'OASIS a été consultée pour comprendre le fonctionnement du modèle publication/abonnement, la notion de broker, les topics et les niveaux de qualité de service. Ce protocole est particulièrement adapté aux systèmes embarqués car il limite la taille des messages et facilite l'ajout de nouveaux consommateurs de données. Les ressources de HiveMQ et EMQX ont également été utilisées pour mieux comprendre les bonnes pratiques de configuration, de test et de sécurisation d'un broker MQTT.

Liens consultés :

- <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- <https://www.hivemq.com/mqtt/>
- <https://www.emqx.com/en/mqtt-guide>

Dashboards et supervision temps réel

Les documentations de Node-RED, Grafana et ThingsBoard ont été étudiées pour identifier les possibilités de visualisation des données. Node-RED permet de construire rapidement des flux de données entre MQTT et une interface graphique. Grafana est adapté aux courbes temporelles et aux historiques de mesure. ThingsBoard propose une plateforme IoT complète intégrant les dashboards, les règles d'alerte et la gestion des entités. Ces références ont aidé à définir les fonctions attendues du dashboard : affichage temps réel, jauges, graphes et alertes.

Liens consultés :

- <https://nodered.org/docs/>
- <https://grafana.com/docs/>
- <https://thingsboard.io/docs/>

Matériel embarqué et capteurs

La documentation technique de l'ESP32 a été utilisée pour justifier le choix du microcontrôleur. L'ESP32 offre une connectivité Wi-Fi intégrée, plusieurs GPIO, des entrées ADC et une compatibilité avec FreeRTOS, ce qui le rend adapté aux prototypes IoT. Les fiches techniques du BMP280 et du DHT11 ont permis d'identifier les plages de mesure, les interfaces électriques et les limites de précision. Les capteurs de son, de vibration, de flamme et de gaz ont été retenus comme modules économiques permettant de démontrer la logique de surveillance, même s'ils nécessitent un étalonnage pour une application industrielle réelle.

Liens consultés :

- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- <https://www.bosch-sensortec.com/products/environmental-sensors/pressure-sensors/bmp280/>
- <https://components101.com/sensors/dht11-temperature-sensor>

Synthèse

Cette recherche bibliographique confirme que le système proposé est cohérent avec les principes de l'IIoT et de la maintenance prédictive. Les travaux sur l'IIoT valident l'organisation en couches du projet, tandis que les travaux sur SCADA montrent l'importance de la supervision, de la fiabilité et de la cybersécurité. Les ressources MQTT et dashboard justifient le choix d'une architecture légère et extensible. Enfin, les documentations matérielles montrent que le prototype est réalisable avec des composants accessibles, tout en gardant des perspectives d'amélioration vers des capteurs plus précis, une communication chiffrée et une supervision plus robuste.